Lesson 14 - Homework

Object-Oriented Programming (OOP) - I

1. Başlamazdan əvvəl — OOP xatırlatması

1.1. OOP, Class və Object nədir?

OOP (Obyekt Yönümlü Proqramlaşdırma) məlumatları və həmin məlumatlarla işləyən funksiyaları birlikdə təşkil etməyə imkan verən proqramlaşdırma yanaşmasıdır.

Məsələn, kitabxanada yüzlərlə kitab var. Hər kitabın adı, müəllifi və səhifə sayı var.

Bunun üçün hər kitabı ayrıca dictionary-də saxlaya bilərik. Amma OOP istifadə edərək bütün kitablar üçün ortaq bir şablon yarada bilərik.

- **Class:** Obyekt yaratmaq üçün şablondur. Məsələn, `Kitab`.
- **Object:** Həmin şablondan yaradılmış konkret nümunədir. Məsələn, `kitab1`.
- **Attribute:** Obyektin saxladığı məlumatdır. Məsələn, kitabın adı, müəllifi.
- **Method:** Class daxilində yazılmış funksiyadır. Məsələn, kitabın məlumatlarını göstərmək.

Sadə bir nümunə:

```
class Kitab:
    def __init__(self, ad, muellif):
        self.ad = ad
        self.muellif = muellif


kitab1 = Kitab("1984", "George Orwell")
kitab2 = Kitab("Dune", "Frank Herbert")

print(kitab1.ad)
print(kitab2.ad)
```

Nəticə:

```
1984
Dune
```

Burada `Kitab` bizim class-ımızdır. `kitab1` və `kitab2` isə bu class-dan yaradılmış iki fərqli obyektdir. Hər obyekt öz məlumatlarını saxlayır.

1.2. `__init__` və `self` nədir?

Bu iki anlayışı başa düşmək OOP üçün çox vacibdir.

`__init__` yeni obyekt yaradılarkən avtomatik çağırılan xüsusi metoddur. Obyektin başlanğıc məlumatlarını təyin etmək üçün istifadə olunur. (Obyekt yaradılarkən yaradılan class-ın adı və mötərizə içərisində __init__ methodu vasitəsi yazdığımız məlumatlar girilir. Yuxarıdakı nümunəni incəliyə bilərsiniz.)

`self` isə hazırda hansı obyektlə işlədiyimizi göstərir.

Yuxarıdakı nümunəyə baxaq:

```
def __init__(self, ad, muellif):
    self.ad = ad
    self.muellif = muellif
```

Biz belə yazdıqda:

```
kitab1 = Kitab("1984", "George Orwell")
```

Python `"1984"` dəyərini `ad` parametrinə, `"George Orwell"` dəyərini isə `muellif` parametrinə ötürür.

Bəs `self.ad = ad` nə deməkdir?

- `ad` — Metoda göndərilən məlumatdır.
- `self.ad` — Həmin obyektin daxilində saxlanılan atributdur.
- `self.ad = ad` — Göndərilən məlumatı obyektin atributunda(yaddaşında) saxlayır.

Beləliklə, sonradan `kitab1.ad` yazaraq həmin məlumata çata bilirik.

Yadda saxla: `self` parametrini obyekt yaradarkən ayrıca göndərmirik. Python onu avtomatik ötürür.

1.3. Method nədir?

Class daxilində yazılan funksiyaya **method** deyilir.

Məsələn, kitabın məlumatlarını göstərən metod yaradaq:

```
class Kitab:
    def __init__(self, ad, muellif):
        self.ad = ad
        self.muellif = muellif

    def melumat_goster(self):
        print(f"Kitab: {self.ad}")
        print(f"Müəllif: {self.muellif}")


kitab1 = Kitab("1984", "George Orwell")
kitab1.melumat_goster()
```

Nəticə:

```
Kitab: 1984
Müəllif: George Orwell
```

`melumat_goster()` bizim metodumuzdur. Onu `kitab1` üzərindən çağırdığımız üçün `self.ad` həmin kitabın adını göstərir.

Metodlar yalnız məlumat göstərmir, həmçinin obyektin məlumatını dəyişə bilər.

Məsələn:

```
class Kitab:
    def __init__(self, sehife):
        self.sehife = sehife

    def sehife_artir(self, miqdar):
        self.sehife += miqdar


kitab1 = Kitab(100)
kitab1.sehife_artir(20)

print(kitab1.sehife)
```

Nəticə: `120`

Burada `self.sehife += miqdar` obyektin əvvəlki səhifə sayına yeni miqdarı əlavə edir.

1.4. `print()` və `return` fərqi

- `print()` məlumatı ekranda göstərir.
- `return` nəticəni geri qaytarır ki, onu başqa yerdə istifadə edə bilək.

Məsələn:

```
def yoxla(self):
    return self.sehife >= 100
```

Bu metod nəticə olaraq `True` və ya `False` qaytarır.

1.5. Class Attribute və Instance Attribute

Instance attribute: Hər obyektin özünə aid məlumatdır. Məsələn, `self.ad`.

Class attribute: Class səviyyəsində saxlanılan, obyektlərin ortaq istifadə edə bildiyi məlumatdır.

```
class Kitab:
    kitabxana = "Merkezi Kitabxana"

    def __init__(self, ad):
        self.ad = ad


kitab1 = Kitab("1984")
kitab2 = Kitab("Dune")

print(kitab1.kitabxana)
print(kitab2.kitabxana)
```

Hər iki obyekt eyni kitabxana adını göstərəcək. Çünki `kitabxana` class attribute-dur, `self.ad` isə hər obyekt üçün ayrıca saxlanılır.

2. Ev tapşırığı — Tələbə İdarəetmə Sistemi

Təsəvvür et ki, High Code tədris mərkəzi üçün sadə tələbə idarəetmə sistemi hazırlayırsan.

Hər tələbənin adı, kursu və balı olacaq. Sistem tələbənin məlumatlarını göstərməli, balını artırmalı və uğurlu olub-olmadığını yoxlamalıdır.

Bütün tapşırıqlar eyni `Telebe` class-ı daxilində yerinə yetirilməlidir.

Tapşırıq 1 — Class və Object yarat

`Telebe` adlı class yarat. `__init__` metodu vasitəsilə aşağıdakı məlumatları qəbul et:

- `ad` — Tələbənin adı
- `kurs` — İştirak etdiyi kurs
- `bal` — Tələbənin balı

Bu məlumatları `self` vasitəsilə obyektin atributlarında saxla.

Test:

```
telebe1 = Telebe("Ayxan", "Python Backend", 45)
telebe2 = Telebe("Tural", "Python Backend", 90)

print(telebe1.ad)
print(telebe2.bal)
```

Gözlənilən nəticə:

```
Ayxan
90
```

İpucu: `Kitab` nümunəsində `self.ad = ad` yazmışdıq. Eyni qayda ilə digər iki atributu da yarat.

Tapşırıq 2 — Tələbənin məlumatlarını göstər

`Telebe` class-ına `melumat_goster()` adlı metod əlavə et.

Metod tələbənin adını, kursunu və balını ekranda göstərməlidir.

Test:

```
telebe1.melumat_goster()
```

Gözlənilən nəticə:

```
Ad: Ayxan
Kurs: Python Backend
Bal: 45
```

İpucu: Metodun ilk parametri `self` olmalıdır. Məlumatları `self.ad`, `self.kurs`, `self.bal` vasitəsilə əldə edə bilərsən.

Tapşırıq 3 — Tələbənin balını artır

`bal_artir(miqdar)` adlı metod yarat.

Qaydalar:

- Əlavə edilən miqdar sıfırdan böyük olmalıdır.
- Yeni bal 100-dən çox ola bilməz.
- Əməliyyat uğurludursa, bal artırılmalı və `True` qaytarılmalıdır.
- Əks halda bal dəyişməməli və `False` qaytarılmalıdır.

Test:

```
telebe1 = Telebe("Ayxan", "Python Backend", 45)

print(telebe1.bal_artir(10))
print(telebe1.bal)

print(telebe1.bal_artir(60))
print(telebe1.bal)
```

Gözlənilən nəticə:

```
True
55
False
55
```

İpucu: Balı artırmaq üçün `self.bal += miqdar` istifadə edə bilərsən. Amma əvvəlcə `if` vasitəsilə miqdarın düzgün olduğunu yoxla.

Tapşırıq 4 — Tələbənin uğurlu olub-olmadığını yoxla

`ugurludur()` adlı metod yarat.

- Bal 51 və ya daha çoxdursa, `True` qaytarsın.
- Bal 51-dən azdırsa, `False` qaytarsın.

Test:

```
telebe1 = Telebe("Ayxan", "Python Backend", 45)
telebe2 = Telebe("Tural", "Python Backend", 90)

print(telebe1.ugurludur())
print(telebe2.ugurludur())
```

Gözlənilən nəticə:

```
False
True
```

İpucu: `self.bal` ilə tələbənin balını yoxla. Bu metod balı dəyişdirməməli, yalnız nəticə qaytarmalıdır.

Tapşırıq 5 — Yanlış balın qarşısını al

`__init__` metoduna yoxlama əlavə et.

Tələbənin balı 0 ilə 100 arasında olmalıdır. Əks halda `ValueError` xətası yaranmalıdır.

Test:

```
telebe = Telebe("Test", "Python Backend", -10)
```

Bu kod `ValueError` yaratmalıdır.

İpucu: Error Handling dərsindən xatırla:

```
if qiymet < 0:
    raise ValueError("Qiymət mənfi ola bilməz.")
```

Sən isə balın həm 0-dan kiçik, həm də 100-dən böyük olmasını yoxlamalısan.

Tapşırıq 6 — Class Attribute əlavə et

Bütün tələbələr High Code tədris mərkəzinə aiddir.

Class daxilində `tehsil_merkezi` adlı class attribute yarat və dəyərini `"High Code"` təyin et.

Test:

```
print(Telebe.tehsil_merkezi)
print(telebe1.tehsil_merkezi)
```

Gözlənilən nəticə:

```
High Code
High Code
```

İpucu: Class attribute `__init__` metodunun daxilində deyil, birbaşa class səviyyəsində yazılır.

3. Final Tapşırıq — Sistemi yoxla

Bütün metodları tamamladıqdan sonra aşağıdakı kodu faylının sonunda işə sal.

```
telebe1 = Telebe("Ayxan", "Python Backend", 45)
telebe2 = Telebe("Tural", "Python Backend", 90)

telebe1.melumat_goster()

print(telebe1.ugurludur())

print(telebe1.bal_artir(10))

print(telebe1.ugurludur())

print(telebe2.bal_artir(20))
print(telebe2.bal)

print(Telebe.tehsil_merkezi)

try:
    telebe3 = Telebe("Test", "Python Backend", -10)
except ValueError as xeta:
    print("Xəta:", xeta)
```

Gözlənilən nəticə:

```
Ad: Ayxan
Kurs: Python Backend
Bal: 45
False
True
True
False
90
High Code
Xəta: Bal 0 ilə 100 arasında olmalıdır.
```

Xəta mesajının mətni fərqli ola bilər. Əsas odur ki, düzgün olmayan bal qəbul edilməsin.

4. Ən çox edilən səhvlər

1. `self` parametrini unutmaq

Yanlış:

```
def melumat_goster():
    print(self.ad)
```

Düzgün:

```
def melumat_goster(self):
    print(self.ad)
```

Metod çağırılarkən Python obyektin özünü avtomatik ötürdüyü üçün ilk parametr kimi `self` yazırıq.

2. Obyektin atributunu yaratmamaq

Yanlış:

```
ad = ad
```

Düzgün:

```
self.ad = ad
```

`self.ad` məlumatın obyektdə saxlanılmasını təmin edir.

3. Metodu çağırarkən mötərizəni unutmaq

Yanlış:

```
telebe1.melumat_goster
```

Düzgün:

```
telebe1.melumat_goster()
```

Metodu işə salmaq üçün `()` istifadə olunur.

Bütün tapşırıqları `lesson_14_hw.py` faylında tamamla, kodu yoxla və GitHub repository-nə commit və push et.

Uğurlar!